Few-Shot Hyperspectral Image Classification Based on Adaptive Subspaces and Feature Transformation

Jing Bai, *Senior Member, IEEE*, Shaojie Huang, Zhu Xiao, *Senior Member, IEEE*, Xianmin Li, Yongdong Zhu, Amelia C. Regan, *Member, IEEE*, and Licheng Jiao, *Fellow, IEEE*

***Abstract*— In the field of hyperspectral image (HSI) classification, deep learning has helped achieve great successes. However, most of these achievements are made with very large amounts of labeled training data. Manual annotation of HSIs is labor intensive and time consuming. In practical HSI classification, there may only be a few labeled samples available. To perform HSI classification with a small number of labeled samples, a new few-shot classification model based on adaptive subspaces and featurewise transformation is proposed in this article. First, we design a 3-D local channel attention residual network to obtain the spatial–spectral features of HSIs. Then, a featurewise transformation strategy is introduced to enhance feature diversity to avoid model overfitting problems and to mitigate the impact of cross-domain problems. Finally, a subspace classifier is implemented to construct different subspace categories based on the embedded features of the limited labeled samples. Classification of an HSI sample is performed using spatial projection and a distance metric. The proposed model is trained using the metalearning mechanism to perform few-shot classification of HSIs. Four public datasets are utilized to construct a sufficient few-shot classification task named episodes for training. The other three public datasets are used to test the proposed model. Experiments show that our proposed method can outperform mainstream small sample HSI classification methods.**

***Index Terms*— 3-D local channel attention residual network (3D-ECA-ResNet), adaptive subspace classifier, featurewise transformation, few-shot learning, hyperspectral image (HSI) classification, metalearning.**

Manuscript received December 23, 2021; accepted January 31, 2022. Date of publication February 9, 2022; date of current version March 24, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 61772401, in part by the Science and Technology on Communication Information Security Control Laboratory, in part by the Funding Project of the Zhejiang Laboratory under Grant 2021LC0AB05, in part by the Key Research and Development Project of Hunan Province of China under Grant 2022GK2020, in part by the Science and Technology Project of the Hunan Provincial Water Resources Department under Grant XSKJ2021000-39, and in part by the Scientific Research Project of the Department of the Natural Resources under Grant 202210. *(Jing Bai and Zhu Xiao contributed equally to this work.) (Corresponding author: Zhu Xiao.)*

Jing Bai, Shaojie Huang, Xianmin Li, and Licheng Jiao are with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, School of Artificial Intelligence, Xidian University, Xi'an 710071, China (e-mail: baijing@mail.xidian.edu.cn; sjhuang_1@stu.xidian.edu.cn; xianminli@stu.xidian.edu.cn; lchjiao@mail.xidian.edu.cn).

Zhu Xiao is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: zhxiao@hnu.edu.cn).

Yongdong Zhu is with the Zhejiang Laboratory, Hangzhou 311121, China (e-mail: zhuyd@zhejianglab.com).

Amelia C. Regan is with the Department of Computer Science and the Institute of Transportation Studies, University of California at Irvine, Irvine, CA 92697 USA (e-mail: aregan@uci.edu).

Digital Object Identifier 10.1109/TGRS.2022.3149947

I. Introduction

With their abundant information in both the spatial and spectral domains, hyperspectral images (HSIs) have been drawing increasing attention from researchers across different industries. HSIs have been implemented in many practical applications, including agriculture management [1], land monitoring, environmental protection [2], and transportation infrastructure monitoring [3]. HSI classification is the fundamental task of HSI analysis. HSI classification entails performing classification on every pixel. Researchers have made different attempts to solve this problem and have achieved a great success, especially with methods using deep learning. The stacked autoencoder in [4] is the first application of deep learning in the field of HSI classification. Subsequently, many researchers have proposed different deep-learning-based methods to tackle the task of HSI classification, such as recurrent neural networks [5], 1-D convolutional neural networks (CNNs) [6], 2-D CNNs [7], [8], and 3-D CNNs [9]. These methods can boost the accuracy of HSI classification to a certain extent. However, achieving great results relies on these algorithms having sufficient labeled data to determine the distribution of the datasets. With the increasing application of hyperspectral sensors, the acquisition of HSI data will become easier, whereas the annotation of HSI data is still labor intensive and time consuming. In reality, there may only be a few labeled samples available in an HSI classification task, and a great number of HSI samples need to be classified. Achieving high classification accuracy with limited samples has become an important challenge to address.

To solve the dilemma of achieving high accuracy with limited labeled training samples, different methods have been proposed. Liu *et al.* [10] proposed a semisupervised method to address HSI classification with limited samples. The basic idea of their method is minimizing the supervised and unsupervised cost functions simultaneously. Liu *et al.* [11] proposed a new feature extraction method to extract well-generalized features from limited HSI data. In addition, some methods based on active learning [12]–[14] were proposed to address HSI classification with small labeled samples. The methods mentioned earlier provided different ideas for tackling the problem of small sample size HSI classification. However, it is still a challenging task for deep-learning-based methods to achieve high accuracy given a limited number of labeled samples.

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

One typical method that deals with HSI classification with a small number of samples is to formulate it as a few-shot learning problem [15]. In few-shot HSI classification, the classification task is formulated as a C-way-K-shot task. Specifically, there are C categories that need to be classified in a task, and K labeled samples from each class are given, and the parameter K is set to a small number. A few-shot learning model will use these $C \times K$ labeled samples to classify other unlabeled samples. These tasks are also called episodes. The model is trained with many episodes in few-shot learning.

Motivated by this, our solution for HSI classification is built on the concept of few-shot learning. Although current few-shot learning-based methods achieve a good performance, they face the following technical challenges.

1) The overfitting problem is more likely to occur and is more serious in HSI few-shot classification than in other methods due to a lack of labeled samples. The main reason behind this is that a feature extraction model with many trainable parameters overlearns the feature representations of the training samples. Therefore, constructing a lightweight feature extraction model with a strong learning ability is a major challenge for HSI few-shot classification.
2) Cross-domain problems [16], [17] emerge if other HSI datasets are involved in the model training phase. Public HSI datasets are typically used to construct training episodes in few-shot HSI classification. The samples in the actual classification task (target domain) may have different categories compared with the available HSI data (source domain). In addition, the sensor type, acquisition conditions, and ground sample distance can differ between the two domains. Accordingly, the embedding feature distribution in the source domain can be different from those in the target domain. This causes cross-domain problems.
3) Mainstream few-shot methods are usually based on metric learning [18] and aim to solve small sample size HSI classification. These methods, which use the mean of extracted embedding features for classification, may not be suitable for HSI few-shot classification. Note that the spectral characteristics of different land cover categories can be similar, and the spectral characteristics of the same land cover category can be different [19]. This phenomenon will degrade the representation ability of embedding features after averaging.

To solve these challenges, we propose a new few-shot classification model based on an adaptive subspace classifier and feature transformation (SSFT). The proposed model consists of three parts: 1) a feature extraction module to obtain high-level features; 2) a featurewise transformation module [20] to generalize the features among different domains; and 3) an adaptive subspace classifier [21] to perform classification. Specifically, our proposed SSFT method tries to overcome the challenges faced by the few-shot HSI classification task from the following aspects.

1) A network called a 3-D local channel attention residual network (3D-ECA-ResNet) is combined with 3-D convolution [22], an efficient channel attention (ECA) module [23], and residual learning [24] to extract HSI embedding features. The utilization of the ECA module can enhance feature extraction of the network while introducing a small number of network parameters.
2) A featurewise transformation module [20] is devised to make the feature representation of 3D-ECA-ResNet more generalized. The featurewise transformation module transforms the feature representation using an affine transformation to change its distribution to prevent the feature extraction model from overlearning the feature representation of the source domain.
3) Instead of only using the mean of embedding features to perform classification in most metric-learning-based methods, we leverage an adaptive subspace classifier [21] to classify HSI samples in our proposed method. The adaptive classifier exploits embedding features to construct the subspace of each class and classifies HSI samples by finding which subspace they belong to. A loss function is designed to ensure that subspaces are as far away from each other as possible. The intervals between the subspaces of different categories will increase adaptively as the training progresses.

In summary, in our proposed model's working process, available HSI datasets with sufficient labeled samples are used to construct many few-shot classification tasks called episodes. During training, the data from these episodes are fed into the model. In each episode, the feature extraction module 3D-ECA-ResNet will extract the limited number of labeled sample's embedding features. The featurewise transformation module will make the distribution of these features more diverse. These features will be used to construct an adaptive subspace classifier. During testing, the limited labeled samples in a few-shot HSI classification task are used to construct subspaces of different classes. The classification of an unlabeled HSI sample is based on its projections onto all the subspaces.

The main contributions of this article can be outlined as follows.

1) A featurewise transformation module is employed to tackle the cross-domain problem in few-shot HSI classification. Experiments show that it can boost the classification accuracy.
2) A robust subspace classifier that can adaptively generate clear classification intervals is introduced to few-shot HSI classification.
3) The proposed few-shot learning-based SSFT method can outperform different kinds of HSI classification methods, including few-shot learning-based methods, semisupervised methods, deep-learning-based methods, and machine learning methods, when only a few labeled samples are available.

The rest of this article is organized as follows. Section II presents the related works. In Section III, the methodology and the proposed SSFT method are described in detail. Then, the experimental results and analysis are presented in Sections IV and V, respectively. Finally, Section VI concludes this article.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

A. Attention Mechanism

Attention modules are designed to capture the important parts of a feature block in deep learning networks. Generally, there are two types of attention mechanisms in deep learning models: channel attention mechanisms and spatial attention mechanisms. Given a feature block with three dimensions (width, height, and channel), which can be the output of a convolution layer, different positions can have a different importance. A channel attention module aims to emphasize the importance of the feature block's different channels by assigning different weights to different channels. A spatial attention mechanism aims to find the important positions in each feature map.

Squeeze-and-excitation network (SE-Net) [25] strengthens important features of the model by modeling the correlation between feature channels. It shows the efficiency of the attention mechanism in extracting features for the first time. Convolutional block attention module (CBAM) [26] implements two attention mechanisms, channel attention and spatial attention, enhancing the model's capability of capturing important features. Concurrent spatial and channel squeeze and channel excitation (scSE) [27] improves the SE-Net network and adopts the mechanisms of channel attention and spatial attention. It increases the accuracy of semantic segmentation tasks. Based on the idea of a nonlocal (NL) mean filter operation in the field of image filtering, an NL network [28] proposes a generalized but simple NL operation operator that can be directly embedded in any network. The NL network can capture long-range dependence in the feature maps.

Essentially, a channel attention module can be formulated as

$$\boldsymbol{\omega} = \sigma(f(g(\boldsymbol{X}))) \tag{1}$$

where $g(\boldsymbol{X}) = (1/WH)\sum_{i=1,j=1}^{W,H} \boldsymbol{X}_{i,j}$ stands for the global average pooling (GAP). The vector $\boldsymbol{\omega}$ is the weight vector of different channels. The σ denotes the sigmoid function. The function f can be viewed as a nonlinear function that computes channel weights. The f in SE-Net [25] is a three-layer neural network. In scSE [27], f represents a two-convolution-layer network.

The attention mechanisms mentioned earlier can enhance the expressive ability of neural networks. However, they generally obtain attention weights through a small neural network, which means that their complexity is relatively high.

B. HSI Classification Based on Metalearning

Deep-learning-based models suffer from overfitting problems when the number of labeled samples is limited. Humans can precisely recognize objects with just a small number of examples, because they can leverage their personal experience. Setting an appropriate model training strategy so that the model can learn like humans can is a new research direction, namely, metalearning [29]. The idea of the metalearning training process is to use the labeled samples from public HSI datasets to construct sufficient few-shot learning tasks and then to use these constructed few-shot learning tasks to train the model and allow it to acquire the ability to solve few-shot learning tasks.

In the field of HSI classification, few-shot learning models based on metalearning have also been studied and tried. Some scholars have proposed the deep few-shot learning model (DFSL) [18], which combines spatial spectrum features with support vector machine (SVM) or nearest neighbor classifiers and uses metalearning methods to train feature extraction models on accurately labeled public hyperspectral datasets to achieve HSI few-shot classification. Rao *et al.* [30] combine the labeled support samples and the query samples to increase training samples. These samples are then fed into a 3-D CNN to extract their spatial and spectral features. A relation network is employed to classify the samples according to the extracted features. Gao *et al.* [31] adopt an induction network and relation network to learn the classwise representation of HSI, rather than its samplewise representation, providing a new idea to solve small sample classification. Another important metric learning model is the prototype network, which uses prototypes to express the common characteristics of classes. The prototype of each class is the mean of embedding vectors from all the samples that belong to the same class. In category prediction, the category of the samples in the query set is the nearest neighbor category in the embedding space. Tang *et al.* [32] proposed using spatial neighborhood correlation to extract the spectral and spatial information of HSIs and used the mean of each class of embedding features as a feature prototype to perform small sample HSI classification. Zhang *et al.* [33] proposed a global prototype network to fully utilize the base class HSI training samples so that the feature prototype of each category can obtain more representational capabilities. Most of the few-shot learning HSI classification methods are metric-based approaches.

The aforementioned methods can achieve great results in few-shot HSI classification tasks. However, they did not consider that the difference between the source domain and target domain of metalearning training will affect the generalization ability of the model. In addition, different sensor types and ground sample distances will also lead to differences in sample distribution between different domains.

C. Cross-Domain Problem in HSI Classification

As discussed in Section I, cross-domain occurs when the categories between the target domain and source domain are different. It is easy for a few-shot classification model to be biased toward the feature distribution of the source domain when a cross-domain problem occurs.

To address the cross-domain learning problem of HSI classification, different methods have been proposed. Basically, these methods can be classified into two categories: fine-tuning-based and domain-adaptation-based approaches. Deng *et al.* [34] proposed a domain adaptation classifier to minimize the bias between the target and source domains in the HSI classification task. Sun *et al.* [16] introduced a method called domain transfer multiple-kernel learning to conquer the influence of different distributions between the target and source domains. Yang and Crawford [35] used similar geometries of images and preserved important HSI data characteristics to avoid the cross-domain problem in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

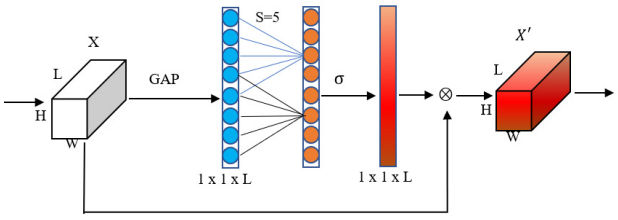


Fig. 1. Diagram of the ECA [23] module. X, the input of the ECA module, can be a feature block generated by a convolution layer. H, W, and L are the height, width, and channel number of the ECA module's input, respectively. GAP means global average pooling. S is the kernel size of the 1-D convolution, determined by L and two hyperparameters.

HSI classification. Wang *et al.* [36] proposed using a neural network to match the distribution in the source domain and extract the manifold embedding to perform domain adaption in HSI classification. The main idea of fine-tuning-based methods is transferring part of the weights of the pretrained network into the target model. Jiao *et al.* [37] proposed fine-tuning-based methods based on spatial distribution prediction. Yang *et al.* [38] proposed a two-branch classifier structure to classify HSIs and adopted a transfer learning strategy to overcome the cross-domain problem. Mei *et al.* [39] trained a CNN to perform classification on the source domain and used the trained CNN's fully connected (FC) feature layers to construct another CNN to classify the samples from the target domain.

The aforementioned methods have made very meaningful explorations on the cross-domain problem in HSI classification. However, the aforementioned domain-adaption-based methods are mainly applied to transfer learning, and are mostly applied to methods based on machine learning. For the fine-tuning-based methods, their performance will drop quickly when the distribution of the two domains differs greatly, and labeled samples from the target domain are limited.

III. Methodology

A. Feature Extraction Model

1) ECA Module: In this paper, an ECA module [23] is adopted to build an embedding feature extraction model. The ECA module utilizes the attention mechanism and remains a relatively simple model structure. Its design philosophy enhances the model's feature extraction ability while retaining a simple structure to avoid overfitting. The ECA module captures the cross-channel interaction of a feature patch in an efficient way. It only considers each channel and its $s-1$ neighbors when computing local cross-channel correlation. It achieves the goal by implementing a 1-D convolution of length s. The ECA module is demonstrated in Fig. 1.

Let $\boldsymbol{X} \in \mathbb{R}^{H\times W\times L}$ denote the input of the ECA module, which can be an output of a convolution layer. H, W, and L are the height, width, and channel of that ECA module input, respectively. The output of the ECA module $\boldsymbol{X}'$ can be computed as

$$\boldsymbol{X}' = \sigma(\boldsymbol{A}g(\boldsymbol{X})) \otimes \boldsymbol{X}. \tag{2}$$

$\boldsymbol{X}'$ has the same size of $\boldsymbol{X}$. The expression $\sigma(\boldsymbol{A}g(\boldsymbol{X}))$ computes the weights of each channel of $\boldsymbol{X}$, where $g(\boldsymbol{X}) = (1/WH)\sum_{i=1,j=1}^{W,H} \boldsymbol{X}_{i,j}$ stands for the GAP. $\boldsymbol{A}$ is a matrix of $L \times L$ size, and the operator σ stands for the sigmoid function. The operator $\otimes$ represents the channelwise multiplication.

The matrix $\boldsymbol{A}$ is used to obtain the local cross-channel interaction of the feature block $\boldsymbol{X}$. $\boldsymbol{A}$ has

$$\begin{bmatrix} a^{1,1} & \cdots & a^{1,s} & 0 & 0 & \cdots & \cdots & 0 \\ 0 & a^{2,2} & \cdots & a^{2,s+1} & 0 & \cdots & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & a^{L,L-s+1} & \cdots & a^{L,L} \end{bmatrix}. \tag{3}$$

From (3), we can see that there are only $s \times L$ parameters in the matrix $\boldsymbol{A}$. After feeding $\boldsymbol{X}$ into the GAP operation $\boldsymbol{y} = g(\boldsymbol{X})$, we can obtain a vector $\boldsymbol{y} = [y_1, \ldots, y_L]^T$. Then, the weight of each channel of the input $\boldsymbol{X}$ is calculated as

$$\omega_i = \sigma\left(\sum_{j=1}^{s} a^{i,j} y_i^j\right), \quad y_i^j \in \Omega_i^s. \tag{4}$$

The value ω_i represents the weight of the ith feature map in the feature cube $\boldsymbol{X}$, and Ω_i^s denotes the set of s adjoining channels of y_i.

Since there are only s nonzero parameters in each row of matrix $\boldsymbol{A}$, y_i and its $s-1$ adjoining channels are only considered when computing the weight of y_i.

To further reduce the parameters of the ECA module, all the channels share the same parameters, that is

$$\omega_i = \sigma\left(\sum_{j=1}^{s} a^{j} y_i^j\right), \quad y_i^j \in \Omega_i^s. \tag{5}$$

Therefore, the channel weight vector $\boldsymbol{\omega} = [\omega_1, \ldots, \omega_L]^T$ can be calculated with a 1-D convolution operation, that is

$$\boldsymbol{\omega} = \sigma(1\text{DCNN}_s(\boldsymbol{y})). \tag{6}$$

There are s parameters involved in the ECA module. The parameter s denotes the cross-channel interaction coverage in the ECA module. If the channel dimension L of $\boldsymbol{X}$ is large, the channel interaction coverage should be larger to capture a larger range of dependence. Therefore, the parameter s should be proportional to the channel dimension L. In addition, the channel dimension L is mostly set to a power of 2. Hence, the relation between the parameter s and the channel dimension L can be determined by an equation, that is

$$C = \phi(s) = 2^{\alpha\times s - b}. \tag{7}$$

For a convolution block with a channel dimension of L, the parameter s is computed as

$$s = \Phi(L) = \left\lceil \left| \frac{\log_2 L}{\alpha} + \frac{b}{\alpha} \right| \right\rceil_{\text{odd}}. \tag{8}$$

The operator $\lceil m \rceil_{\text{odd}}$ indicates the nearest odd number of m. In our work, the parameters α and b are set to 2 and 1.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

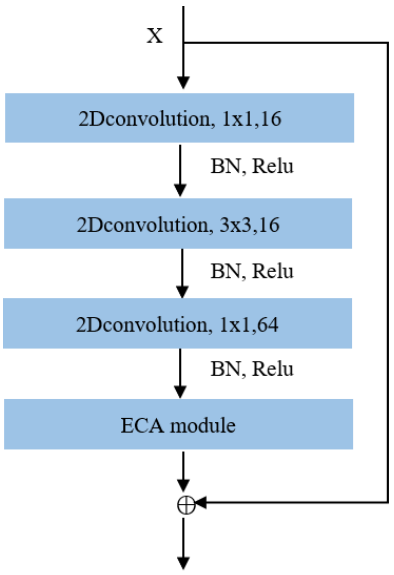


Fig. 2. Diagram of the ECA-residual-block module.

2) ECA-Residual Block: To enhance our model's ability to extract the spatial and spectral features of HSIs, residual blocks are also exploited in our feature extraction module. Specifically, we add the ECA module to the basis residual block. A feature block after passing through three convolution layers will be fed into an ECA module. Then, the output will be added to an identity of the original feature block. The fused module is called ECA-residual block. The diagram of the ECA-residual block is shown in Fig. 2. The introduction of a residual block avoids the vanishing and exploding gradient problem after stacking too many layers in our feature extraction model.

3) 3-D Convolution: In our proposed method, HSI data are presented as 3-D patches. To utilize the spatial and spectral features of HSI data more efficiently, we exploit 3-D convolution layers in our feature extraction model. Common 2-D convolution layers only convolve the feature cubes in two dimensions. The spatial and spectral characteristics of HSI data are captured separately, and the joint interaction information of the HSI data spectral and spatial dimensions is underutilized. The 3-D convolution layers convolve the feature cube in three dimensions. Therefore, 3-D convolution can capture more information about HSI data than 2-D convolution.

The overall structure of our feature extraction model is shown in Fig. 3. There are two 3-D convolution layers and four ECA-residual blocks in the feature extraction model. GAP stands for global average pooling. The parameters of the ECA-residual blocks are different. These details will be described in Section IV-B.

B. Featurewise Transformation Module

A featurewise transformation module is proposed to tackle the cross-domain problem in few-shot HSI classification and make the extracted feature more generalized in our work. In actual HSI classification applications, the newly acquired HSIs (target domain) have only a few labeled samples, while other HSI datasets (source domain) have adequate labeled samples; the categories in two domains could be different. This may lead to inconsistent embedding feature distributions in the two domains. In addition, using other HSI datasets with adequate labeled samples to train the classification model may result in overfitting. The feature transform module is connected to the end of the feature extraction module. It changes the extracted feature distribution by implementing an affine transformation to produce a more diverse feature distribution. Specifically, there are two hyperparameters in the featurewise transformation module, $\boldsymbol{\theta}_\gamma \in \mathbb{R}^{1\times1\times L}$ and $\boldsymbol{\theta}_\beta \in \mathbb{R}^{1\times1\times L}$, indicating the Gaussian distributions' standard deviations. The affine transformation parameters β and γ are sampled from the specific Gaussian distributions.

After obtaining the feature representation $\boldsymbol{Z} \in \mathbb{R}^{H\times W\times L}$ extracted by the feature extraction model, we should then obtain the two parameters of the affine transformation, the bias term β and the scaling term γ using

$$\beta \sim N\big(\mathbf{0}, \text{softplus}\big(\boldsymbol{\theta}_\beta\big)\big), \quad \gamma \sim N\big(\mathbf{1}, \text{softplus}\big(\boldsymbol{\theta}_\gamma\big)\big) \tag{9}$$

$$\text{softplus}(x) = \log\big(1 + e^x\big). \tag{10}$$

Then, we can obtain the transformed feature representation $\hat{\boldsymbol{Z}} \in \mathbb{R}^{H\times W\times L}$ by

$$\hat{z}_{h,w,l} = \gamma_l \times z_{h,w,l} + \beta_l \tag{11}$$

where $\hat{z}_{h,w,l} \in \hat{\boldsymbol{Z}}$ and $z_{h,w,l} \in \boldsymbol{Z}$.

The featurewise transformation module is adopted in the training phase to transform the embedding feature of the HSI base datasets. Experiments reveal that this feature transformation strategy can increase HSI classification accuracy in the few-shot situation.

C. Adaptive Subspace Classifier

Based on the idea of an adaptive subspace classifier, we assume that the feature representations belong to a specific class forming a subspace within a high-dimensional space. The adaptive subspace classifier models points by subspaces $\{\boldsymbol{Z}_i\}_{i=1}^N$. The subspace $\boldsymbol{Z}_i$ can be represented by a basis $\boldsymbol{B}_i \in \mathbb{R}^{D\times n}$, $n \le D$, with $\boldsymbol{B}_i^T\boldsymbol{B} = \boldsymbol{I}_n$. The feature representations extracted by the feature extraction model are utilized to form the adaptive subspace classifier. Matrix decomposition is applied to obtain the class subspace basis vectors.

The feature extraction module θ can be viewed as a function f_θ: $\boldsymbol{X} \to \mathbb{R}^D$ that maps the input HSI image patch $\boldsymbol{X}$ to a D-dimensional vector. Let $\boldsymbol{X}_c = \{\boldsymbol{x}_{c,1}, \ldots, \boldsymbol{x}_{c,k}\}$ be a class-specific set of size k. The sample set $\boldsymbol{X}_c$ is fed into the feature extraction module θ to obtain the extracted features, that is

$$\boldsymbol{Z_c} = \big[f_\theta\big(\boldsymbol{x}_{c,1}\big), \ldots, f_\theta\big(\boldsymbol{x}_{c,k}\big)\big]. \tag{12}$$

In the proposed SSFT method, the extracted features pass through the featurewise transformation module. The transformed features can then be expressed as

$$\hat{\boldsymbol{Z}}_{\boldsymbol{c}} = \big[T\big(f_\theta\big(\boldsymbol{x}_{c,1}\big)\big), \ldots, T\big(f_\theta\big(\boldsymbol{x}_{c,k}\big)\big)\big] \tag{13}$$

where the function T stands for the feature transform operation.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

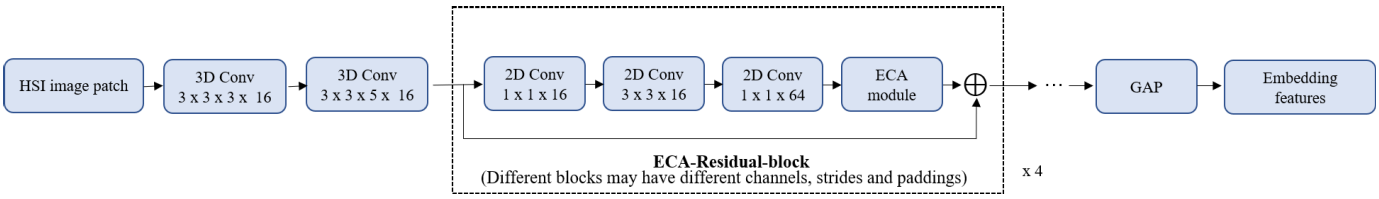


Fig. 3. Diagram of the proposed feature extraction model 3D-ECA-ResNet.

Then, the sample set $\boldsymbol{X}_c$ encoded by the feature extraction module and featurewise transformation module can be expressed as

$$\hat{\boldsymbol{X}}_{\boldsymbol{c}} = \left[T\left(f_\theta\left(\boldsymbol{x}_{c,1}\right)\right) - \mu_c, \ldots, T\left(f_\theta\left(\boldsymbol{x}_{c,k}\right)\right) - \mu_c\right] \quad (14)$$

where $\mu_c = (1/K)\sum_{\boldsymbol{x}_i \in \boldsymbol{X}_c} T(f_\theta(\boldsymbol{x}_i))$ is the mean of transformed feature vectors of the samples in category C. After obtaining $\hat{\boldsymbol{X}}_c$, the subspace that represents category C can be constructed by applying singular value decomposition (SVD) to $\hat{\boldsymbol{X}}_c$, that is

$$\hat{\boldsymbol{X}}_{\boldsymbol{c}} = \boldsymbol{B}_{\boldsymbol{c}} \boldsymbol{\Sigma} \boldsymbol{V}_{\boldsymbol{c}}^{\boldsymbol{T}}. \quad (15)$$

The first n dimensions of $\boldsymbol{B}_c$ are selected to form a truncation matrix $\boldsymbol{P}_c$. The matrix $\boldsymbol{P}_c$ is viewed as the basis of the subspace of category C in the high-dimensional space.

Classifying a sample means locating the subspace to which the sample belongs, which is the subspace to which the embedding feature vector of the sample has the shortest projection distance among all the subspaces. Specifically, let $\boldsymbol{q}_j$ be a sample that needs to be classified. The projection distance between the sample $\boldsymbol{q}_j$'s embedding feature vector to $\boldsymbol{P}_c$ can be calculated as

$$d_c\left(\boldsymbol{q}_j\right) = -\left\|\left(\boldsymbol{I} - \boldsymbol{M}_c\right)\left(f_\theta\left(\boldsymbol{q}_j\right) - \boldsymbol{\mu}_c\right)\right\|^2 \quad (16)$$

where $\boldsymbol{M}_c = \boldsymbol{P}_c \boldsymbol{P}_c^T$. The probability of a sample $\boldsymbol{q}_j$ that belongs to class C is defined as

$$p_{c,q_j} = p\left(c \| \boldsymbol{q}_j\right) = \frac{\exp\left(d_c\left(\boldsymbol{q}_j\right)\right)}{\sum_{c_i} \exp\left(d_{c_i}\left(\boldsymbol{q}_j\right)\right)}. \quad (17)$$

D. Metalearning

A metalearning training mechanism is exploited to train the proposed few-shot HSI classification model. In the training process of metalearning, the model is trained episode by episode. An episode T_i is a simulation of a few-shot classification task, which is usually formulated as a C-way-K-shot-classification problem. There are two sets of data in a C-way-K-shot-classification task, query set S and support set Q. There are C class labels in total, and there are K labeled data for each class in the support set. Let $S = \{(x_{1,1}, c_{1,1}), (x_{1,2}, c_{1,2}), \ldots, (x_{C,K}, c_{C,K})\}$ be the denotation of the support set, where $x_{i,j}$ denotes class i's jth sample and $c_{i,j} \in \{1, \ldots, C\}$. In the support set, the number of samples is small, and these samples are the only labeled examples we have for training the proposed model. In query set Q, M query data need to be classified. The query set can be denoted as $Q = \{q_1, \ldots, q_M\}$. A query set and a support set form an episode T_i. In the metalearning paradigm, given a set of episodes, an algorithm is improving its learning ability if its performance after each episode improves with experience and with the number of episodes.

In this paper, to address the HSI few-shot classification problem using metalearning, public HSI datasets with sufficient labeled samples are used to construct episodes. In each episode, when training the few-shot classification model, we sample C classes of HSI image patches and K patches per class as the support set. In addition, M HSI image patches are sampled from the public dataset as the query set. In this way, a classification task that is similar to the few-shot classification task is formed. In the support set, HSI samples are labeled. These labeled samples are used to extract the subspace of each class. At the end of each episode, the parameters of the model are trained to maximize the accuracy of the samples from query set Q.

E. SSFT Model

Fig. 4 displays a diagram of the proposed SSFT model. For each hyperspectral remote sensing image $\boldsymbol{I} \in \mathbb{R}^{H \times W \times L}$, to avoid the redundancy of spectral information, a principal component analysis (PCA) strategy is adopted to reduce the dimensions of the image. After performing dimensionality reduction, we obtain $\boldsymbol{I} \in \mathbb{R}^{H \times W \times L'}$. In the spatial dimension, we select the 27×27 neighborhood of each pixel to construct the input data cube $\boldsymbol{x} \in \mathbb{R}^{27 \times 27 \times L'}$. In our experimental setting, L' is set to 5, which means that we use the PCA operation to reduce the channels of HSIs to 5. In the feature extraction model, multiple ECA-residual blocks will be constructed to increase the depth of the model so that it can extract the spatial–spectral information effectively. Meanwhile, to avoid the feature extraction model overfitting the base class data feature distribution, a featurewise transformation module is added, followed by the output layer of the feature extraction model during the training phase to increase the diversity of the features distribution. A metalearning mechanism is utilized to train the model. During the training phase, the proposed HSI few-shot classification model SSFT is not trained to learn the embedding feature of the base class data; instead, it is trained to learn how to learn. Specifically, during the training phase, C class samples and K per class are randomly selected to forge a support set. The labels of the C selected base class data are reassigned from 0 to $C - 1$. In other words, C_0 indicates the base class data that are reassigned to label 0. The extracted feature representation that belongs to the same category cluster subtracts the average of that category's embedding features. After that, $\tilde{\boldsymbol{X}}_{\boldsymbol{c}}$ is obtained. SVD is applied

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

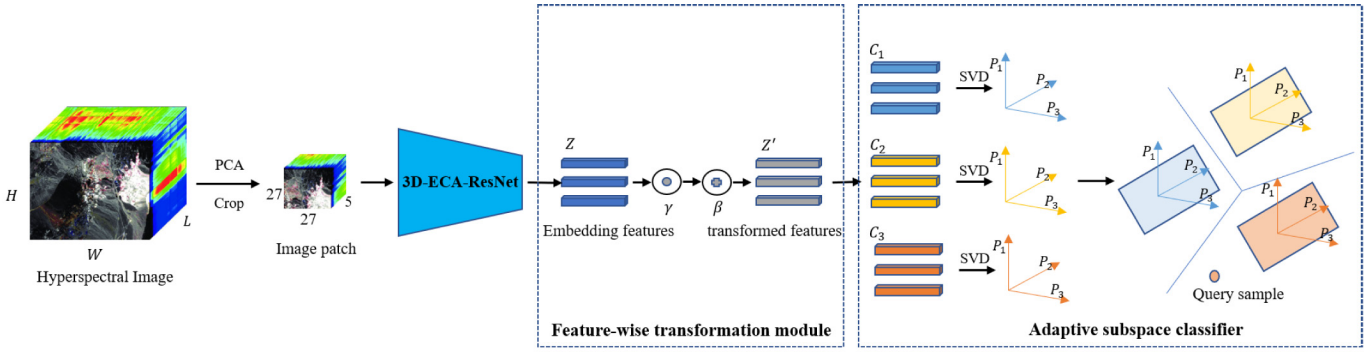


Fig. 4. Architecture of our proposed model. H, W, and L are the height, width, and number of channels of the HSI, respectively. After applying the PCA operation on the HSI, the number of channels becomes 5. We select every pixel's 27×27 spatial neighborhood as an input sample. The input sample, the image patch, will be fed into the backbone network 3D-ECA-ResNet. The featurewise transformation module transforms the embedding out of the backbone network. The transformed features will be used to construct a subspace classifier.

TABLE I

PARAMETERS OF THE PROPOSED FEATURE EXTRACTION MODEL

Layer name	Output size	Kernel size and number	stride	padding
input	$27 \times 27 \times 5 \times 1$	N/A	N/A	N/A
3D_conv_1	$27 \times 27 \times 5 \times 16$	$3 \times 3 \times 3 \times 16$	(1,1,1)	(1,1,1)
3D_conv_2	$27 \times 27 \times 1 \times 16$	$3 \times 3 \times 5 \times 16$	(1,1,1)	(1,1,0)
squeeze	$27 \times 27 \times 16$	N/A	N/A	N/A
ECA_Residual_block_1	$14 \times 14 \times 64$	$\begin{bmatrix} 1 \times 1 \times 16 \\ 3 \times 3 \times 16 \\ 1 \times 1 \times 64 \\ ECA \end{bmatrix}$	$\begin{bmatrix} (1,1) \\ (2,2) \\ (1,1) \end{bmatrix}$	$\begin{bmatrix} (0,0) \\ (1,1) \\ (0,0) \end{bmatrix}$
ECA_Residual_block_2	$14 \times 14 \times 64$	$\begin{bmatrix} 1 \times 1 \times 16 \\ 3 \times 3 \times 16 \\ 1 \times 1 \times 64 \\ ECA \end{bmatrix}$	$\begin{bmatrix} (1,1) \\ (1,1) \\ (1,1) \end{bmatrix}$	$\begin{bmatrix} (0,0) \\ (1,1) \\ (0,0) \end{bmatrix}$
ECA_Residual_block_3	$7 \times 7 \times 80$	$\begin{bmatrix} 1 \times 1 \times 20 \\ 3 \times 3 \times 20 \\ 1 \times 1 \times 80 \\ ECA \end{bmatrix}$	$\begin{bmatrix} (1,1) \\ (2,2) \\ (1,1) \end{bmatrix}$	$\begin{bmatrix} (0,0) \\ (1,1) \\ (0,0) \end{bmatrix}$
ECA_Residual_block_4	$7 \times 7 \times 80$	$\begin{bmatrix} 1 \times 1 \times 20 \\ 3 \times 3 \times 20 \\ 1 \times 1 \times 80 \\ ECA \end{bmatrix}$	$\begin{bmatrix} (1,1) \\ (1,1) \\ (1,1) \end{bmatrix}$	$\begin{bmatrix} (0,0) \\ (1,1) \\ (0,0) \end{bmatrix}$
Global Average Pooling	$1 \times 1 \times 80$	N/A	N/A	N/A

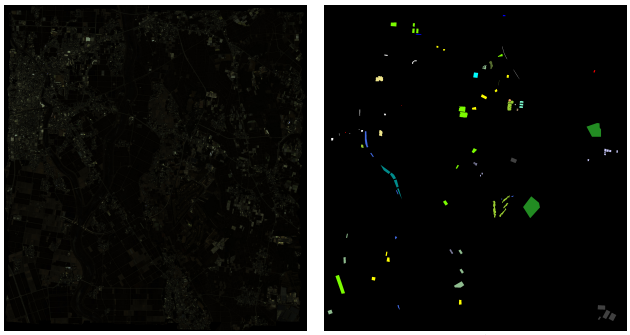
to $\tilde{\boldsymbol{X}}_c$, and the truncated matrix represented by the first N singular values is taken as the subspace basis vectors for the specific category. After obtaining the subspace of each category, classification can be performed by computing the distance between the query sample's feature and its projection onto the subspaces.

The objective of the proposed few-shot classification model is to learn effective feature representations to generate more discriminative subspaces and enhance the overall classification performance. Specifically, the projection distance of subspaces should be maximized during the training phase. Given the basis of two subspaces $\boldsymbol{P}_i$ and $\boldsymbol{P}_j$, the projection distance between two subspaces can be calculated as

$$\delta_p^2(\boldsymbol{P}_i, \boldsymbol{P}_j) = \left\| \boldsymbol{P}_i \boldsymbol{P}_i^T - \boldsymbol{P}_i \boldsymbol{P}_i^T \right\|_F^2 = 2n - 2\left\| \boldsymbol{P}_i^T \boldsymbol{P}_j \right\|_F^2. \quad (18)$$

From the aforementioned equation, we can tell that maximizing the projection distance between two subspaces means minimizing the term $\|\boldsymbol{P}_i^T \boldsymbol{P}_j\|_F^2$. Therefore, the loss function is defined as

$$-\frac{1}{M} \sum_c \log(p_{c,q}) + \lambda \sum_{i \neq j} \left\| \boldsymbol{P}_i^T \boldsymbol{P}_j \right\|_F^2. \quad (19)$$



(a) (b)

Fig. 5. Chikusei dataset. (a) Pseudocolor image. (b) Ground-truth map.

The first part of the loss function is the cross-entropy loss. The training process of the SSFT method is shown in Algorithm 1. The parameters of our feature extraction model are illustrated in Table I.

IV. EXPERIMENTAL ANALYSIS

A. Experimental Datasets

The proposed method aims to address the HSI few-shot classification task. In reality, newly acquired HSI remote

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

TABLE II
GENERAL INFORMATION ABOUT THE FOUR BASE DATASETS

	Chikusei	Botswanna	Houston	KSC
Spatial size	2517×2335	1476×256	349×1905	512×614
Spectral range	368-1018	400-2500	380-1050	400-2500
No. of bands	128	145	144	176
GSD	2.5	30	2.5	18
Sensor type	Hyperspec-VNIR-C	EO-1	ITRES-CASI1500	AVIRIS
Areas	Chikusei	Botswana	Houston	Florida
No. of classes	19	14	15	13

Algorithm 1 Training Process for SSFT. Public HSI Datasets Are Used to Construct Episodes. In Each Episode, There Are Two Sets of Data, a Support Set S and a Query Set Q

Input:
Support set S and query set Q from each episode T_i
Initial feature extraction model Θ_0
Output:
Trained feature extraction model Θ;
for each episode T_i **do**
for each class C in support set S **do**
Obtain the extracted features $\boldsymbol{Z_c}$ using Equation (12);
Obtain the transformed features $\hat{\boldsymbol{Z}}_{\boldsymbol{c}}$ using Equation (13);
Compute $\hat{\boldsymbol{X}}_{\boldsymbol{c}}$ using Equation (14);
Compute the singular value matrix of $\hat{\boldsymbol{X}}_{\boldsymbol{c}}$ using $\hat{\boldsymbol{X}}_{\boldsymbol{c}} = \boldsymbol{B_c \Sigma V_c^T}$;
Truncate the orthogonal matrix $\boldsymbol{B}_c$, select the first n dimensions as the subspace basis vectors of category C;
for each sample $\boldsymbol{q}_j$ in query set Q **do**
Calculate $d_c(\boldsymbol{q}_j)$ using Equation (16);
end for
end for
Calculate L_t using Equation (19);
Use the gradient of L_t to update the parameters of feature extraction model Θ;
end for
return Trained feature extraction model Θ;

sensing image datasets may only have a few labeled samples, while the existing accurately labeled public HSI datasets could be sufficient. The proposed model is trained in a metalearning fashion to take advantage of existing HSI datasets and let the model learn how to learn. Eventually, the model can effectively predict the given HSI data relying on very few labeled samples. In our experiments, the Chikusei dataset [40], Botswana dataset, the Kennedy Space Center (KSC) dataset, and Houston dataset are utilized as the base class datasets and their pseudocolor images and ground-truth maps are shown in Figs. 5–8. General information on the four HSI datasets is illustrated in Table II. The HSI data from the Pavia University (PU) dataset, Indian Pines (IP) dataset, and Salinas (SA) dataset are utilized as testing samples. The testing dataset information is illustrated in Table III.

B. Network Architecture of SSFT

To fully capture the spectral and spatial information of HSI data, a feature extraction model shown in Fig. 3 is proposed.

TABLE III
BRIEF INFORMATION ABOUT THE TESTING DATASETS

	Pavia University	Indian Pines	Salinas
Spatial size	610×340	145×145	512×217
Spectral range	430-860	400-2500	400-2500
No. of bands	103	200	204
GSD	1.3	20	3.7
Sensor type	ROSIS	AVIRIS	AVIRIS
Areas	Pavia	Indiana	California
No. of classes	9	16	16

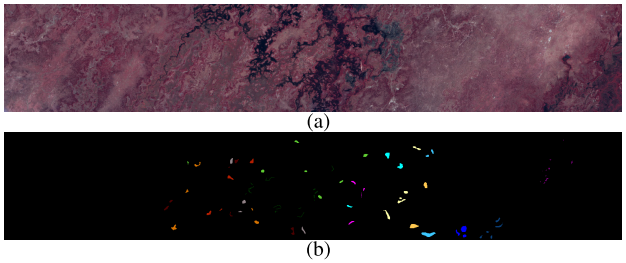

(a)
(b)

Fig. 6. Botswanna dataset. (a) Pseudocolor image. (b) Ground-truth map.

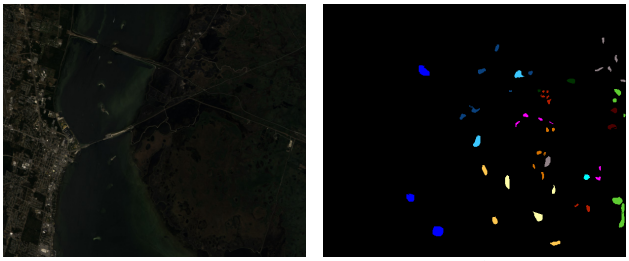

(a) (b)

Fig. 7. KSC dataset. (a) Pseudocolor image. (b) Ground-truth map.

The parameters of the model are illustrated in Table I. To describe the model more accurately, we divide it into two stages. In the first stage of the model, two 3-D convolution layers are exploited to extract the spectral–spatial information of the HSI image patches. In the second stage of the model, four ECA-residual blocks with different channels, strides, and paddings are used to form the main part of the feature extraction model. Batch normalization and the rectified linear unit (ReLU) activation function are adopted to increase the feature's nonlinearity and avoid feature distribution shifting after every convolution layer. The hyperparameters of the ECA modules α and b are set to 2 and 1, respectively, to widen the dependence range of high-dimensional channels and narrow the dependence range of the low-dimensional

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

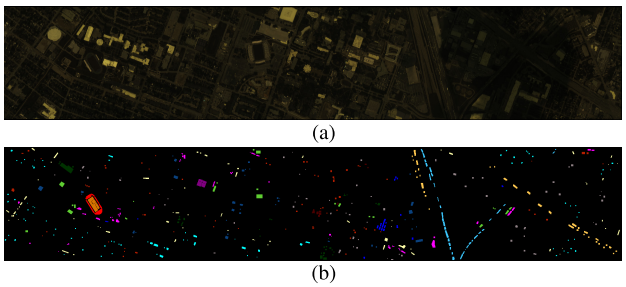


Fig. 8. Houston dataset. (a) Pseudocolor image. (b) Ground-truth map.

channels. This will make the cross-channel interaction efficient and rapid. During training, the featurewise transformation module is connected to the end of the feature extraction model to avoid overfitting on the base class samples and make the distribution of the extracted feature more diverse, increasing the generalization of the model. The hyperparameters of the featurewise module θ_γ and θ_β are set to 0.3 and 0.5, respectively. After a GAP operation is performed, the output-extracted feature is a vector of 80 dimensions. In the adaptive subspace classifier, the features belonging to the same category will be stacked, and truncated SVD (TSVD) will be applied. The truncated matrix represented by the first $K-1$ singular values will be viewed as the subspace basis vectors of this category, and K is the number of shots in the support set of each episode.

C. Training Data Settings for SSFT

A metalearning mechanism is adopted during training the model. Four public HSI datasets were exploited to train the feature extraction model. The images from the four HSI datasets are used as the base class samples. The 27×27 neighborhoods of each pixel are selected in an original HSI image. PCA is applied to the HSI patches, and the first five components are selected. Therefore, the size of each HSI patch is $27 \times 27 \times 5$. After reassigning the labels, there are 77 categories in the base class sample set in total. Considering the uneven distribution of the number of each category, the categories with fewer than 200 samples were abandoned. Ultimately, there are 54 categories in the base class dataset. Then, we randomly sample 200 samples from each class as the base class samples. Three HSI datasets—the PU dataset, SA dataset, and IP dataset—are used for testing. There are at most 16 different categories in each of the three datasets. Therefore, in each episode, the number of randomly selected categories for training needs to be greater than 16. In the training phase, 25 classes are randomly selected to construct one episode for training. The number of query samples of each class is 15. The number of labeled samples of each class in the support set is set to 5. In each epoch, there will be 100 episodes of HSI samples, which means that there will be 100 metalearning operations in each epoch. The learning rate is set to 0.01 initially, the attenuation factor of the learning rate is set to 0.5, and the learning rate decays every 50 epochs; the model will go through 300 epochs.

D. Comparison Methods

To validate the efficiency of the proposed SSFT model, we selected a number of mainstream small sample HSI classification models for comparison. They include the deep few-shot learning models, that is, DFSL-SVM [18] and DFSL-NN [18], semisupervised convolutional neural network (SS-CNN) [10], graph convolutional network (GCN) [41], 3DCNN [42], and SVM with grid search [43]. These methods cover most of the approaches that deal with HSI small sample classification, including few-shot learning methods, semisupervised models, graph convolution models, common neural networks, and traditional machine learning methods. To prove the effectiveness of the featurewise transformation module, the proposed method without a featurewise transformation module, SSFT without feature transformation (SSW/OFT), is also tested.

Non-metalearning methods, SVM, 3D-CNN, SS-CNN, and GCN, are trained on target domain datasets directly with different numbers of labeled samples and tested on all unlabeled samples. For fairness of comparison, our SSFT method's base dataset design (discussed in Section IV-C) is the same as DFSL-SVM and DFSL-NNs [18]. Therefore, for metalearning methods, DFSL-SVM and DFSL-NN, their base dataset is also composed of four HSI datasets (Chikusei, Botswana, Houston, and KSC). Two hundred samples per class from these four datasets are selected to form the base dataset. However, the sample size of DFSL-SVM and DFSL-NN is $9 \times 9 \times 100$ while training and testing these two methods according to their original paper. DFSL-SVM and DFSL-NN are also tested on three target datasets with different numbers of labeled samples.

We have tried our best to make each comparison method competitive. The model structures of the comparison methods we chose are the structures that can achieve the highest accuracy in their original papers. For hyperparameters that affect the model accuracy, such as the neighborhood size of the HSI samples, we referenced their original papers and conducted verification experiments to select the best hyperparameter settings. In terms of training, different learning rates, optimizers, etc., have also been attempted. In addition, we trained the comparison methods with enough epochs to ensure that they converged.

E. Evaluation Metrics

We use the overall accuracy (OA), average accuracy (AA), and kappa coefficient. These three indicators are commonly used in HSI classification tasks. These three indicators can be calculated from the confusion matrix. A classification confusion matrix is shown as follows:

$$U = \begin{bmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,N_c} \\ u_{2,1} & u_{2,2} & \cdots & u_{2,N_c} \\ \vdots & \vdots & \cdots & \vdots \\ u_{N_c,1} & u_{N_c,2} & \cdots & u_{N_c,N_c} \end{bmatrix}. \tag{20}$$

In (20), $u_{i,j}$ represents the number of HSI samples belonging to the jth category that are classified into the ith category. N_c represents the total number of categories. From the confusion matrix, we can calculate the OA, AA, and kappa

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
OA (%) OF DIFFERENT METHODS ON IP DATASET (AVERAGE OF TEN RUNS ± STANDARD DEVIATION)

methods	shot5	shot10	shot15	shot20
SVM	47.09±4.78	55.99±2.15	59.62±3.21	61.92±1.84
3D-CNN	53.63±5.00	62.67±3.83	68.16±3.59	72.18±2.27
SS-CNN	52.31±2.34	58.47±3.34	63.11±6.40	64.35±6.17
GCN	51.56±1.92	55.98±4.71	58.90±5.74	68.17±1.49
DFSL-NN	66.09±2.99	74.15±2.93	78.10±0.93	80.93±1.20
DFSL-SVM	66.47±3.35	74.66±2.17	79.11±3.01	80.64±2.19
SSW/OFT	66.90±2.65	74.23±3.16	77.26±1.91	79.13±2.09
SSFT	**68.66±2.84**	**76.54±1.28**	**83.01±1.34**	**85.21±1.58**

TABLE V
OA (%) OF DIFFERENT METHODS ON PU DATASET (AVERAGE OF TEN RUNS ± STANDARD DEVIATION)

methods	shot5	shot10	shot15	shot20	shot25
SVM	64.40±5.03	70.26±4.25	73.95±2.56	78.03±1.98	79.18±2.14
3D-CNN	65.76±6.96	69.92±4.71	78.05±3.04	76.83±3.10	81.94±1.88
SS-CNN	48.94±6.29	58.09±6.22	64.45±2.89	67.39±2.79	68.87±3.34
GCN	68.38±3.65	69.38±4.34	71.14±4.46	72.47±4.65	78.89±3.14
DFSL-NN	76.94±5.47	82.63±5.37	86.15±2.79	88.01±2.49	89.68±1.57
DFSL-SVM	73.78±5.99	78.50±4.51	85.42±5.66	87.64±4.77	89.63±6.23
SSW/OFT	71.53±2.60	83.11±2.87	83.80±1.72	86.11±1.24	88.04±1.94
SSFT	**81.37±2.21**	**86.79±2.14**	**87.55±2.29**	**90.57±1.82**	**91.77±0.85**

TABLE VI
OA (%) OF DIFFERENT METHODS ON SA DATASET (AVERAGE OF TEN RUNS ± STANDARD DEVIATION)

methods	shot5	shot10	shot15	shot20	shot25
SVM	78.48±3.50	84.04±2.36	86.11±1.89	86.14±1.61	86.70±1.68
3D-CNN	82.02±2.86	86.22±1.45	87.28±2.18	86.66±2.18	88.57±1.98
SS-CNN	70.70±2.66	75.36±3.37	81.41±3.28	80.74±2.50	81.97±3.77
GCN	81.12±2.07	82.31±2.12	83.14±2.36	83.83±2.49	86.97±1.11
DFSL-NN	87.71±3.07	90.75±1.33	91.31±1.45	92.24±0.40	93.04±0.46
DFSL-SVM	86.17±2.72	88.28±2.58	88.17±2.34	89.54±2.60	93.07±1.31
SSW/OFT	86.81±2.28	90.99±1.91	94.02±1.04	95.53±0.10	95.64±1.03
SSFT	**91.21±1.65**	**92.94±1.76**	**94.88±1.18**	**96.10±0.80**	**96.57±0.85**

coefficient as follows:

$$\mathrm{OA} = \frac{1}{N}\sum_{j=1}^{N_c} u_{jj} \tag{21}$$

$$\mathrm{AA} = \frac{\sum_{j=1}^{N_c} \frac{u_{jj}}{u_{+j}}}{N_c} \tag{22}$$

$$\mathrm{Kappa} = \frac{N\sum_{j=1}^{N_c} u_{jj} - \sum_{j=1}^{N_c} u_{j+}u_{+j}}{N^2 - \sum_{j=1}^{N_c} u_{j+}u_{+j}}. \tag{23}$$

In (21)–(23), N denotes the total number of test samples. u_{jj} denotes the number of samples correctly classified for category j. u_{+j} represents the sum of column j, which is the sample number of category j. u_{j+} represents the sum of row j, which is the number of samples predicted as category j.

V. RESULTS AND DISCUSSION

A. Classification Accuracy

We take 5, 10, 15, 20, and 25 labeled samples per class for analysis from the PU dataset and the SA dataset. For the IP HSI dataset, we only take 5, 10, 15, and 20 labeled samples per class, because the class "Oats" only has 20 samples present in this dataset. To avoid the randomness of model prediction, all experiments are verified ten times. We choose the OA and the standard deviation of OA as the evaluation criteria. The experimental results are illustrated in Tables IV–VI. Figs. 9–11 are the classification maps of each dataset with 25 labeled samples selected for each class (20 labeled samples for the IP dataset). The precision of each class with only 25 labeled samples per class given for training (20 for the IP dataset) in different datasets is also explored, and the results are shown in Tables VII–IX.

From these experimental results, we can obtain the following phenomena.

1) From Tables IV–VI, we can ascertain that the proposed SSFT method outperforms all the comparison methods in terms of OA in all the cases of given labeled sample numbers. In the case of five labeled samples per class, our method exceeds the OA of DFSL-NN, which is also a metalearning method, by 2% on the IP dataset, 5% on the PU dataset, and 4% on the SA dataset. This shows the competitiveness of our method in few-shot classification.
2) The classification accuracy of the methods based on metalearning is higher than that of other methods.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

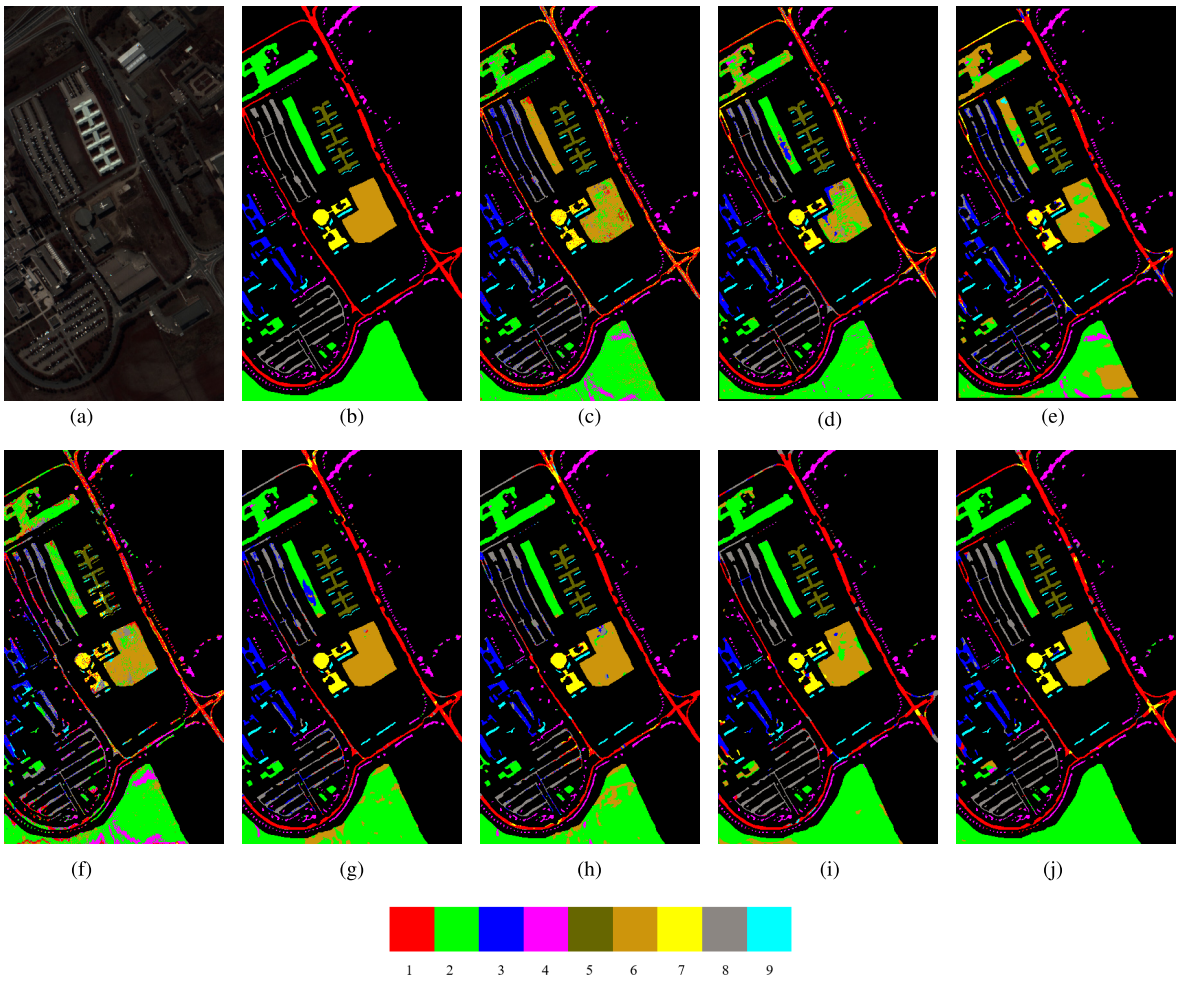


Fig. 9. Classification maps of different methods with 25 labeled samples per class on the PU dataset. 1—Asphalt, 2—Meadows, 3—Gravel, 4—Trees, 5—Painted metal sheets, 6—Bare soil, 7—Bitumen, 8—Self-blocking bricks, and 9—Shadows. (a) Pseudocolor image. (b) Ground-truth map. (c) SVM. (d) 3-D-CNN. (e) SS-CNN. (f) GNN. (g) DFSL-NN. (h) DFSL-SVM. (i) SSW/OFT. (j) SSFT.

TABLE VII
CLASSIFICATION PRECISION OF EACH CLASS WITH 25 LABELED SAMPLES PER CLASS ON PU DATASET

Class Name	SVM	3D-CNN	SS-CNN	GCN	DFSL-NN	DFSL-SVM	SSW/OFT	SSFT
Asphalt	92.71±2.18	94.97±1.72	94.27±1.37	**95.65±1.52**	95.63±1.06	93.73±1.43	89.52±2.91	95.13±1.64
Meadows	91.46±1.43	92.85±2.26	90.56±3.69	93.36±1.39	98.27±0.82	**98.84±0.84**	98.62±0.51	98.18±0.81
Gravel	63.26±6.53	76.14±9.24	46.57±6.73	64.31±8.03	73.78±6.50	81.66±6.67	**82.98±4.21**	77.86±4.78
Trees	76.16±8.63	83.78±9.63	78.74±17.54	70.60±12.21	**93.39±5.12**	87.69±10.71	57.06±5.62	85.85±4.01
Painted metal sheets	97.37±1.80	97.87±1.58	99.83±0.22	99.54±0.30	99.88±0.09	**99.99±0.05**	88.96±1.61	96.59±2.32
Bare Soil	55.14±5.13	62.51±7.71	47.71±5.74	56.25±7.76	78.87±6.79	79.69±16.11	84.77±9.79	**85.21±5.07**
Bitumen	48.72±6.61	60.40±5.03	43.30±8.25	54.56±6.70	66.65±8.96	**80.74±9.87**	78.38±8.28	76.78±5.40
Self-Blocking Bricks	79.19±2.41	84.78±3.16	75.85±6.96	72.32±2.93	77.17±5.38	80.17±7.06	**89.88±4.02**	87.44±2.78
Shadows	**99.87±0.11**	99.31±0.93	93.38±7.21	96.51±3.36	96.14±2.13	85.74±8.23	75.05±6.70	85.82±3.89
OA	79.18±2.14	81.94±1.88	68.87±3.34	78.89±3.14	89.68±1.57	89.63±6.23	88.04±1.94	**91.77±0.85**
AA	78.21±1.42	75.29±0.90	67.10±1.87	78.14±1.87	86.64±1.46	87.74±3.47	82.80±1.99	**87.95±1.07**
Kappa	73.29±2.49	76.92±2.22	61.77±3.30	73.19±3.65	86.56±1.98	86.71±7.39	84.47±2.40	**89.21±1.09**

In the comparison methods, DFSL-SVM, DFSL-NN, and our SSFT method are based on metalearning, GCN and SS-CNN are based on semisupervised learning, and SVM and 3D-CNN are general supervised learning. Among all metalearning methods, DFSL-SVM has the worst OA. However, when compared with other non-metalearning methods, its results are still competitive. In Tables IV–VI, in the shot 5 case, the OA of DFSL-SVM exceeds 3D-CNN by 13% on the IP dataset, by 8% on the PU dataset, and by 4% on the SA dataset. This shows that the metalearning methods have more advantages in few-shot classification problems.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

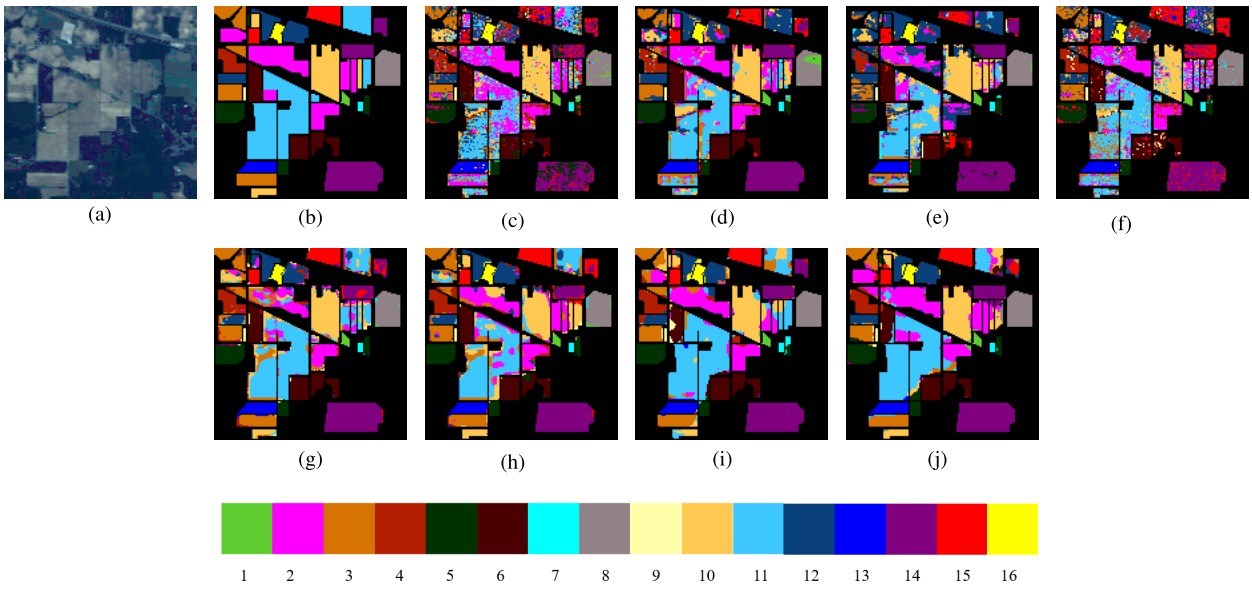


Fig. 10. Classification maps of the different methods with 20 labeled samples per class on the IP dataset. 1—Alfalfa, 2—Corn-notill, 3—Corn-mintill, 4—Corn, 5—Grass-pasture, 6—Grass-trees, 7—Grass-pasture-mowed, 8—Hay-windrowed, 9—Oats, 10—Soybean-notill, 11—Soybean-mintill, 12—Soybean-clean, 13—Wheat, 14—Woods, 15—Buildings-grass-trees-drives, and 16—Stone-steel-towers. (a) Fake color image. (b) Ground-truth map. (c) SVM. (d) 3-D-CNN. (e) SS-CNN. (f) GCN. (g) DFSL-NN. (h) DFSL-SVM. (i) SSW/OFT. (j) SSFT.

TABLE VIII
CLASSIFICATION PRECISION OF EACH CLASS WITH 20 LABELED SAMPLES PER CLASS ON THE IP DATASET

Class Name	SVM	3D-CNN	SS-CNN	GCN	DFSL-NN	DFSL-SVM	SSW/OFT	SSFT
Alfalfa	25.46±5.89	45.43±14.38	71.83±9.00	54.39±12.83	78.17±10.93	**79.69±9.79**	50.05±10.11	56.53±12.40
Corn-notill	55.87±4.90	71.86±6.25	77.96±8.60	68.95±4.71	78.69±4.54	73.73±8.70	81.12±7.47	**81.97±4.32**
Corn-mintill	40.26±8.47	55.33±7.80	54.22±19.61	48.68±7.59	70.04±5.87	74.33±7.33	**90.15±2.08**	83.14±5.89
Corn	36.84±4.38	47.12±8.66	51.31±15.47	36.26±6.49	54.48±3.91	58.26±11.90	72.60±9.97	**75.09±6.39**
Grass-pasture	66.39±9.72	89.52±6.28	73.53±17.25	87.89±4.37	96.31±2.44	**97.59±2.07**	53.49±10.07	85.35±6.48
Grass-trees	89.42±2.39	91.73±2.85	91.93±3.17	93.63±1.95	**99.38±0.69**	99.37±0.66	97.48±1.51	87.06±2.66
Grass-pasture-mowed	25.05±8.47	27.19±16.11	31.66±19.06	18.70±8.09	53.38±12.73	**76.05±14.44**	68.06±7.34	58.62±14.32
Hay-windrowed	98.18±0.65	98.33±2.36	99.66±0.75	98.46±0.72	**100.00±0.00**	**100.00±0.00**	59.15±12.08	97.01±2.72
Oats	6.12±1.59	10.02±7.07	4.90±2.07	0.00±0.00	25.78±3.97	43.51±17.68	**72.71±6.38**	30.13±7.38
Soybean-notill	52.29±5.13	58.73±7.09	48.92±20.59	59.52±7.48	61.52±4.90	61.51±5.69	72.95±11.06	**77.57±5.81**
Soybean-mintill	69.93±4.70	79.84±1.98	72.69±10.83	75.97±3.23	89.15±2.83	87.70±4.59	84.09±9.43	**92.56±2.01**
Soybean-clean	40.72±5.34	53.85±8.07	47.71±10.62	46.48±5.7	70.75±4.89	70.40±6.93	**86.09±4.32**	77.85±6.97
Wheat	80.01±6.58	92.18±6.23	88.38±3.73	79.59±6.74	93.32±5.92	**97.08±4.12**	45.00±9.69	87.25±5.22
Woods	92.68±1.61	96.14±2.09	94.92±1.84	94.80±1.34	**99.05±0.73**	98.51±2.20	86.53±5.11	95.85±2.11
Buildings-Grass-Trees-Drives	48.99±7.44	68.56±4.20	73.96±8.60	50.22±7.00	73.24±2.18	72.07±6.68	30.97±7.68	**81.81±6.73**
Stone-Steel-Towers	93.34±7.20	93.24±6.30	88.52±4.66	77.48±12.70	87.88±5.10	**93.36±3.34**	69.95±8.88	71.15±7.85
OA	61.92±1.84	72.18±2.27	61.35±8.16	68.17±1.49	80.93±1.20	80.64±2.19	79.13±2.09	**85.21±1.58**
AA	57.60±1.47	63.40±1.50	62.40±1.69	61.93±1.58	76.95±1.21	76.45±1.25	70.02±2.22	**77.44±2.11**
Kappa	57.30±1.93	68.73±2.46	57.11±8.36	64.28±1.60	78.57±1.35	78.22±2.41	76.54±2.29	**83.31±1.74**

3) The classification OA of all methods will increase, as the number of given labeled samples increases. This phenomenon is particularly obvious in the IP dataset. In Table IV, when the labeled samples increased from 5 per class to 20 per class, the OA of SSFT increased by 17%, and the OA of DFSL-SVM also increased by 14%.
4) The classification maps on different datasets of SSFT are the closest to the ground-truth classification map. In Fig. 9, we can see that SSFT performs better than other methods on the category "Meadows." In Fig. 10, SSFT is more accurate in the category "Grass-pasture-mowed." In Fig. 11, SSFT is closer to the ground truth in the category "Grapes-untrained" compared with other methods.
5) Looking at the accuracy of each category in Tables VIII and IX, our proposed SSFT method performs better in some categories that are different from those of other methods. For example, distinct categories are the class "Corn" in the IP dataset and "Vinyard untrained" in the SA dataset.

B. Examination of the Impact of Ablation

To verify whether the ECA module and the feature transformation module have an impact on the classification results, we explored the effect of removing the ECA module and the feature transformation module from the model both separately and together, and the experimental results are shown

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

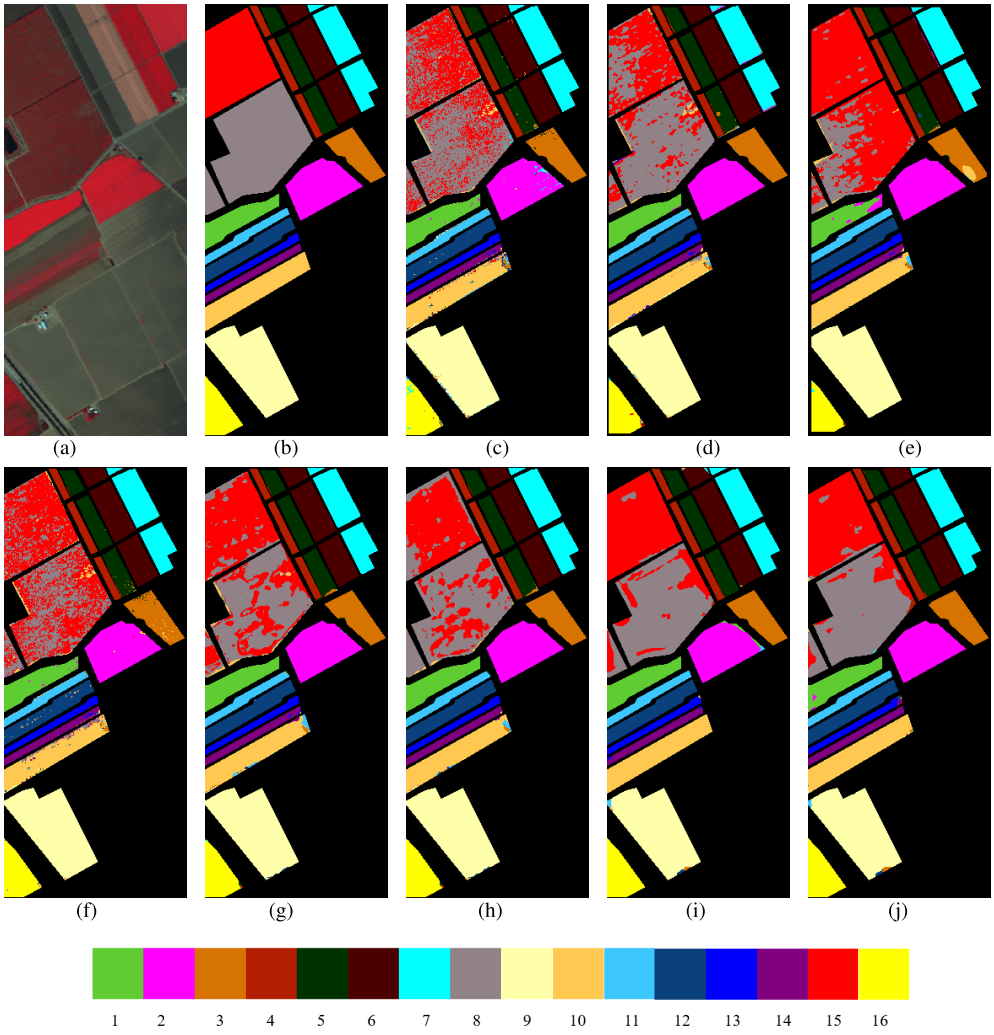


Fig. 11. Classification maps of different methods with 25 labeled samples per class on the SA dataset. 1—Brocoli_green_weeds_1, 2—Brocoli_green_weed_2, 3—Fallow, 4—Fallow_rough_plow, 5—Fallow_smooth, 6—Stubble, 7—Celery, 8—Grapes_untrained, 9—Soil_vinyard_develop, 10—Corn_senesced_green_weeds, 11—Lettuce_romaine_4wk, 12—Lettuce_romaine_5wk, 13—Lettuce_romaine_6wk, 14—Lettuce_romaine_7wk, 15—Vinyard_untrained, and 16—Vinyard_vertical_trellis. (a) Pseudocolor image. (b) Ground-truth map. (c) SVM. (d) 3-D-CNN. (e) SS-CNN. (f) GCN. (g) DFSL-NN. (h) DFSL-SVM. (i) SSW/OFT. (j) SSFT.

in Table X. From the experimental results, we can see that removing the ECA module or the feature transformation module will reduce the classification accuracy of the model. Specifically, removing the feature transformation module will have a great impact on the classification accuracy, which shows that the feature transformation module can indeed make the generalization ability of the model stronger.

Experiments on three datasets have also been implemented to explore the impact of subspace dimensions n on classification results. Table XI shows that the subspace dimensionality n does not affect the classification results much when n is greater than 10. These experiments are conducted with 25 labeled samples per class (20 for the IP dataset). Each experiment was performed ten times to mitigate the randomness of model prediction. The classification results are evaluated with OA, AA, and kappa criteria and their standard deviation of ten experiments. Beside, we also compare the complexity of all observed methods and the experimental results are shown in Table XII.

In addition, we also conducted an ablation experiment on the datasets used during metatraining. The experimental results are shown in Table XIII. We select one of the four datasets (Chikusei, Botswana, Houston, and KSC) to train our model and test the accuracy of the model on the three datasets (SA, PU, and IP) when there are only five labeled samples in each class. From the experimental results, the model trained with the Botswana dataset alone is more competitive than the model trained with other datasets alone. The model trained on the four datasets at the same time has the highest accuracy.

C. Computational Complexity Analysis

To analyze the computational complexity of each method, we compared the training and testing times of all observed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
CLASSIFICATION PRECISION OF EACH CLASS WITH 25 LABELED SAMPLES PER CLASS ON THE SA DATASET

Class Name	SVM	3D-CNN	SS-CNN	GCN	DFSL-NN	DFSL-SVM	SSW/OFT	SSFT
Brocoli_green_weeds_1	97.82±1.86	**100.00±0.00**	81.89±25.29	99.73±0.66	99.99±0.01	**100.00±0.00**	99.80±0.22	99.26±0.62
Brocoli_green_weeds_2	98.67±0.40	99.16±0.44	96.22±8.07	99.25±0.14	**99.89±0.12**	99.82±0.26	99.51±0.38	99.39±0.83
Fallow	92.33±2.45	96.23±1.05	95.31±1.04	93.46±2.42	96.30±0.81	97.40±1.28	94.35±6.21	**97.92±1.53**
Fallow_rough_plow	97.79±0.46	98.21±1.40	97.44±1.95	96.54±1.24	98.68±0.91	**99.10±0.90**	98.79±1.56	98.45±1.37
Fallow_smooth	97.70±0.78	97.67±1.34	96.47±6.35	97.47±1.43	98.25±0.42	**98.77±0.67**	95.25±2.61	94.22±5.28
Stubble	99.90±0.06	99.7±0.26	**100.00±0.00**	99.99±0.01	99.99±0.02	**100.00±0.00**	91.70±5.95	95.12±1.88
Celery	97.57±1.25	99.69±0.26	98.60±1.43	99.12±0.93	**99.89±0.10**	99.60±0.71	84.67±4.61	89.25±4.84
Grapes_untrained	74.20±3.53	84.44±3.83	78.58±10.90	74.34±3.91	89.39±1.90	88.43±3.82	**100.00±0.00**	98.10±1.75
Soil_vinyard_develop	99.13±0.27	99.35±0.53	**99.79±0.20**	99.27±0.21	99.66±0.13	99.67±0.15	99.21±1.30	99.64±0.75
Corn_senesced_green_weeds	83.57±3.03	90.03±4.10	86.81±5.34	85.36±4.37	**94.81±0.95**	92.76±2.91	87.90±5.82	91.46±3.30
Lettuce_romaine_4wk	87.95±5.63	92.57±2.07	93.87±2.93	91.99±3.17	92.35±3.44	94.97±3.25	**97.56±0.96**	96.97±1.91
Lettuce_romaine_5wk	96.30±1.06	96.85±2.07	97.32±5.24	95.77±1.88	98.51±0.86	**99.47±0.55**	98.94±1.11	99.00±0.70
Lettuce_romaine_6wk	94.87±2.74	93.22±3.27	99.43±0.68	93.54±2.03	**100.00±0.00**	**100.00±0.00**	99.27±0.75	99.67±0.38
Lettuce_romaine_7wk	93.96±2.72	91.21±5.22	98.31±0.85	91.63±3.02	97.13±1.22	**99.32±0.62**	95.66±2.61	95.70±2.21
Vinyard_untrained	60.82±7.81	67.13±8.38	58.89±10.89	61.17±6.42	73.78±1.90	78.25±6.13	99.93±0.17	**99.96±0.06**
Vinyard_vertical_trellis	97.19±1.34	98.51±1.71	99.25±0.49	98.71±0.75	**99.98±0.05**	89.48±15.54	98.25±1.23	99.21±0.69
OA	86.70±1.68	88.57±1.98	81.97±3.77	86.97±1.11	93.04±0.46	93.07±1.31	95.64±1.03	**96.57±0.85**
AA	91.90±0.83	88.50±0.50	88.21±1.25	92.17±0.43	96.16±0.36	96.06±0.85	96.30±0.84	**97.08±0.71**
Kappa	85.21±1.85	87.33±2.16	80.04±4.16	85.50±1.21	92.27±0.52	92.30±1.44	95.15±1.14	**96.18±0.94**

TABLE X
OA (%) OF DIFFERENT METHODS ON DIFFERENT DATASETS (AVERAGE OF TEN RUNS ± STANDARD DEVIATION, "SSFTW/OECA" MEANS REMOVING THE ECA MODULE, "SSW/OFT" MEANS REMOVING THE FEATURE TRANSFORMATION MODULE, AND "SSFTW/OECA&FT" MEANS REMOVING THE ECA MODULE AND FEATURE TRANSFORMATION MODULE)

Datasets	Methods	Shot5	Shot10	Shot15	Shot20	Shot25
PU	SSFTW/OECA	74.87±5.02	85.41±2.57	86.57±1.79	87.84±1.73	89.15±1.90
	SSW/OFT	71.53±2.60	83.11±2.87	83.80±1.72	86.11±1.24	88.04±1.94
	SSFTW/OECA&FT	71.63±5.48	84.08±2.51	83.36±2.99	86.93±1.42	87.67±0.71
	SSFT	**81.37±2.21**	**86.79±2.14**	**87.55±2.29**	**90.57±1.82**	**91.77±0.85**
SA	SSFTW/OECA	87.05±2.68	91.21±0.94	93.91±0.99	95.40±0.73	95.90±0.51
	SSW/OFT	86.81±2.28	90.99±1.91	94.02±1.04	95.53±0.10	95.64±1.03
	SSFTW/OECA&FT	86.33±2.10	88.64±1.36	93.44±2.19	94.24±1.09	93.81±1.03
	SSFT	**91.21±1.65**	**92.94±1.76**	**94.88±1.18**	**96.10±0.80**	**96.57±0.85**
IP	SSFTW/OECA	67.88±2.52	75.82±2.95	80.83±1.04	81.73±1.42	-
	SSW/OFT	66.90±2.65	74.23±3.16	77.26±1.91	79.13±2.09	-
	SSFTW/OECA&FT	63.38±4.65	74.39±3.01	78.69±1.92	80.99±1.59	-
	SSFT	**68.66±2.84**	**76.54±1.28**	**83.17±1.27**	**85.21±1.58**	-

TABLE XI
EXPERIMENTS WITH DIFFERENT SUBSPACE DIMENSIONS n OF SUBSPACE CLASSIFIER, 25 LABELED SAMPLES PER CLASS WHILE TESTING FOR PU AND SA DATASETS AND 20 FOR IP DATASET

	Iindian Pines			Salinas			Pavia University		
n	OA	AA	Kappa	OA	AA	Kappa	OA	AA	Kappa
2	75.97±1.36	64.08±1.50	73.09±1.46	94.39±0.69	94.99±0.49	93.76±0.77	84.69±2.48	80.38±2.94	84.21±2.12
4	78.77±2.39	68.70±0.93	76.19±2.63	95.43±0.72	96.55±0.76	94.92±0.80	88.79±1.74	83.77±1.67	85.47±2.17
6	81.70±0.66	70.32±1.08	79.41±0.72	96.68±0.89	97.30±1.06	96.31±0.99	90.20±1.61	85.67±1.70	87.24±2.00
8	83.14±1.00	72.81±1.05	80.99±1.14	96.57±0.70	97.03±0.57	96.19±0.78	91.43±1.55	86.81±0.84	88.77±1.98
10	83.34±0.88	72.82±0.73	81.24±0.99	96.61±1.25	**97.48±0.51**	96.23±1.39	91.78±1.08	87.59±0.94	88.72±1.32
12	83.14±2.68	72.32±2.88	81.02±2.94	96.36±0.75	97.14±0.52	95.95±0.84	90.91±1.18	87.81±1.56	88.11±1.48
14	84.15±0.97	73.64±1.03	82.10±1.09	96.11±0.45	97.11±0.34	95.68±0.50	**91.79±2.05**	87.53±2.46	88.69±2.58
16	85.39±0.92	77.13±1.02	83.37±1.05	96.58±0.75	97.35±0.46	96.20±0.83	90.27±0.72	85.62±0.77	87.34±0.94
18	85.43±0.96	77.73±2.77	83.34±1.04	96.32±0.70	97.07±0.37	95.91±0.78	89.30±2.27	84.27±2.45	86.13±2.83
20	85.69±1.37	77.22±1.94	83.49±1.42	96.64±0.65	97.24±0.36	96.26±0.71	90.38±2.18	85.72±1.45	87.46±2.70
22	**85.95±1.05**	78.36±1.21	**83.80±1.17**	**96.82±0.49**	97.46±0.39	**96.46±0.54**	91.01±1.21	87.10±1.86	88.23±1.52
24	85.38±1.03	**78.95±1.56**	83.24±1.12	96.57±0.85	97.08±0.71	96.18±0.94	91.77±0.85	**87.95±1.07**	**89.21±1.09**

methods. The experimental results are shown in Table XII. All experiments were run on the same device, which was a Windows laptop with an AMD Ryzen 7 5800H CPU with 16-GB RAM and an RTX 3070 GPU. In addition, we also compared all the deep-learning-based method's numbers of floating-point operations (FLOPs) and model parameters. The experimental results are also shown in Table XII. The metalearning-based methods (DFSL-SVM, DFSL-NN, and SSFT) are trained on the base class dataset (source domain) and tested on the target domain datasets (SA, PU,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

TABLE XII

COMPUTATIONAL TIME (SECONDS), FLOPS, AND PARAMETERS FOR DIFFERENT METHODS ON DIFFERENT DATASETS (FIVE LABELED SAMPLES PER CLASS GIVEN)

Dataset	Methods	SVM	3D-CNN	SS-CNN	GCN	DFSL-NN	DFSL-SVM	SSFT
SA	Training time(s)	1	39	380	248	368	368	142
	Inference time(s)	0.1	3	6	0.5	1	3	12
	FLOPs	N/A	4537096	23790540	55272960	41143324	41143324	3467248
	Params	N/A	45196	4833820	29224	41964	41964	81640
PU	Training time(s)	1	26	634	135	368	368	142
	Inference time(s)	0.1	3	8	0.3	0.8	1.5	5
	FLOPs	N/A	2310266	15861952	1149979829	41143324	41143324	3467248
	Params	N/A	38476	4360032	15191	41964	41964	81640
IP	Training time(s)	1	35	56	38	368	368	142
	Inference time(s)	0.1	0.5	0.8	0.3	0.2	1.7	2.7
	FLOPs	N/A	4449686	23466849	535058112	41143324	41143324	3467248
	Params	N/A	45196	4810209	28704	41964	41964	81640

TABLE XIII

OA (%) OF DIFFERENT METHODS ON DIFFERENT DATASETS (AVERAGE OF TEN RUNS ± STANDARD DEVIATION, FIVE LABELED SAMPLES PER CLASS, THE SSFT (CHIKUSEI) METHOD USES THE CHIKUSEI DATASET DURING METATRAINING; THE SSFT (BOTSWANA) METHOD USES THE BOTSWANA DATASET DURING METATRAINING; THE SSFT (HOUSTON) METHOD USES THE HOUSTON DATASET DURING METATRAINING; THE SSFT (KSC) METHOD USES THE KSC DATASET DURING METATRAINING; AND THE SSFT (FOUR) METHOD USES THE CHIKUSEI, BOTSWANA, HOUSTON, AND KSC DATASETS DURING METATRAINING)

dataset	SSFT(Chikusei)	SSFT(Botswana)	SSFT(Houston)	SSFT(KSC)	SSFT(four)
SA	86.91±2.21	87.33±2.10	86.75±1.70	88.61±1.49	91.21±1.65
IP	66.44±2.14	68.97±2.49	64.93±2.29	59.09±3.13	68.66±2.84
PU	69.36±3.08	74.63±3.70	69.63±7.81	64.31±2.11	81.37±2.21

and IP) with five labeled samples per class. Other methods are trained and tested on the three target domain datasets (SA, PU, and IP) with five labeled samples per class during training and all the unlabeled samples during testing.

The experimental results show that our SSFT method maintains relatively smaller FLOPs and a shorter training time while achieving competitive classification accuracy.

D. Discussion

From the experimental results in Tables IV–VI, we can see that the methods using different training modes have different classification performances. The methods (DFSL-SVM, DFSL-NN, and SSFT) that adopted the metalearning strategy can outperform other methods. For the methods that use the ordinary supervised training mode, SVM and 3D-CNN, their classification performance is poor. This is mainly because the supervised learning method will very easily overfit when the number of labeled training samples is very small. As the labeled samples increase from 5 to 25 per class, the overfitting problem is alleviated, and the OA also increases. For the methods using semisupervised training, SS-CNN and GCN, their classification OA is also inferior to the method using metalearning. This may be because there are too few labeled samples, and semisupervised methods cannot capture much information that can be used for classification. Metalearning methods can outperform other methods when the number of labeled samples for training is small. This is because the training process of metalearning is proposed for solving small sample problems.

Another important reason why our SSFT method can outperform other methods is that we propose a solution to the cross-domain problem in HSI few-shot classification. Other metalearning-based methods, DFSL-NN and DFSL-SVM, do not consider that the inconsistency between the source domain and the target domain will affect the generalization ability of the model. In addition, the samples of the source domain and the target domain will experience a large difference in their distribution due to the different sensor types and ground sample distances. From Table X, we can observe that removing the featurewise transformation module from the SSFT method has the greatest impact on classification OA. This shows that the feature conversion module can indeed improve the classification accuracy of our model.

In addition, from the ablation experimental results, the utilization of the ECA module can also boost the classification accuracy of SSFT. From Table XII, our SSFT model's number of FLOPs is less than other deep-learning-based methods. Our SSFT method not only achieves high classification accuracy but also is simpler in operation than other methods. This shows that our backbone network is designed properly. In addition, in our SSFT method, the subspace classifier is trained with the backbone network. The subspace classifier can guide the backbone network to generate embedding features that are more suitable for constructing subspaces. However, in other metalearning-based methods, such as DFSL-SVM and DFSL-NN, their metric-based classifiers [SVM and K-nearest neighbor (KNN)] are separated from their backbone during training.

VI. CONCLUSION

In this article, we proposed a new few-shot HSI classification method called the adaptive SSFT. In the proposed method, a feature extraction model that integrates 3-D convolution, an ECA attention module, and residual learning is proposed to extract the spatial and spectral features of HSIs effectively. In few-shot HSI classification, overfitting can be a major problem when there are few labeled samples. The introduction

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

of an ECA module can enhance the model's feature extraction ability while maintaining a relatively simple model structure with few parameters. Fewer parameters mean that overfitting is less likely. A featurewise transformation module is adopted during training to address the cross-domain problem in few-shot HSI classification. Experiments show that the featurewise transformation module can better generalize features, which increases classification accuracy. Then, an adaptive subspace classifier is designed to perform the classification of an HSI sample by finding the subspace to which its embedding feature vector belongs. The few-shot model is trained by metalearning. Four public HSI datasets are employed to construct base class datasets, and the other three HSI datasets are used to test the proposed model. The metalearning training strategy can avoid model overfitting when only a few labeled samples are available. Experiments show that our SSFT method can outperform mainstream small sample HSI classification methods.

REFERENCES

[1] L. Liang *et al.*, "Estimation of crop LAI using hyperspectral vegetation indices and a hybrid inversion method," *Remote Sens. Environ.*, vol. 165, pp. 123–134, Aug. 2015.

[2] X. Yang and Y. Yu, "Estimating soil salinity under various moisture conditions: An experimental study," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2525–2533, May 2017.

[3] O. Saprykin, A. Fedoseev, and T. Mikheeva, "Recognition of urban transport infrastructure objects via hyperspectral images," in *Proc. Int. Conf. Vehicle Technol. Intell. Transp. Syst.*, 2016, pp. 203–208.

[4] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[5] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[6] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jul. 2015, Art. no. 258619.

[7] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, Jun. 2015.

[8] B. Liu, X. Yu, A. Yu, and G. Wan, "Deep convolutional recurrent neural network with transfer learning for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 12, no. 2, 2018, Art. no. 026028.

[9] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[10] B. Liu, X. Yu, P. Zhang, X. Tan, A. Yu, and Z. Xue, "A semi-supervised convolutional neural network for hyperspectral image classification," *Remote Sens. Lett.*, vol. 8, no. 9, pp. 839–848, Sep. 2017.

[11] B. Liu, X. Yu, P. Zhang, A. Yu, Q. Fu, and X. Wei, "Supervised deep feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 1909–1921, Nov. 2017.

[12] S. Rajan, J. Ghosh, and M. M. Crawford, "An active learning approach to hyperspectral data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 4, pp. 1231–1242, Apr. 2008.

[13] D. Tuia, F. Ratle, F. Pacifici, M. F. Kanevski, and W. J. Emery, "Active learning methods for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 7, pp. 2218–2232, Jul. 2009.

[14] M. M. Crawford, D. Tuia, and H. L. Yang, "Active learning: Any value for classification of remotely sensed data?" *Proc. IEEE*, vol. 101, no. 3, pp. 593–608, Mar. 2013.

[15] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.

[16] Z. Sun, C. Wang, H. Wang, and J. Li, "Learn multiple-kernel SVMs for domain adaptation in hyperspectral data," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 5, pp. 1224–1228, Sep. 2013.

[17] Y. Guo, H. Cao, J. Bai, and Y. Bai, "High efficient deep feature extraction and classification of spectral-spatial hyperspectral image using cross domain convolutional neural networks," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 1, pp. 345–356, Jan. 2019.

[18] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[19] S. Jia, S. Jiang, Z. Lin, N. Li, M. Xu, and S. Yu, "A survey: Deep learning for hyperspectral image classification with few labeled samples," *Neurocomputing*, vol. 448, pp. 179–204, Aug. 2021.

[20] H.-Y. Tseng, H.-Y. Lee, J.-B. Huang, and M.-H. Yang, "Cross-domain few-shot classification via learned feature-wise transformation," 2020, *arXiv:2001.08735*.

[21] C. Simon, P. Koniusz, R. Nock, and M. Harandi, "Adaptive subspaces for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 4136–4145.

[22] S. Ji, W. Xu, M. Yang, and K. Yu, "3D convolutional neural networks for human action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 221–231, Jan. 2013.

[23] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11531–11539.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[25] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[26] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 3–19.

[27] A. G. Roy, N. Navab, and C. Wachinger, "Recalibrating fully convolutional networks with spatial and channel 'squeeze and excitation' blocks," *IEEE Trans. Med. Imag.*, vol. 38, no. 2, pp. 540–549, Feb. 2019.

[28] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7794–7803.

[29] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," in *Proc. ICLR*, Toulon, France, Mar. 2017, pp. 1–11.

[30] M. Rao, P. Tang, and Z. Zhang, "Spatial–spectral relation network for hyperspectral image classification with limited training samples," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 12, pp. 5086–5100, Dec. 2019.

[31] K. Gao, W. Guo, X. Yu, B. Liu, A. Yu, and X. Wei, "Deep induction network for small samples classification of hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3462–3477, 2020.

[32] H. Tang, Y. Li, X. Han, Q. Huang, and W. Xie, "A spatial–spectral prototypical network for hyperspectral remote sensing image," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 1, pp. 167–171, Jan. 2020.

[33] C. Zhang, J. Yue, and Q. Qin, "Global prototypical network for few-shot hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4748–4759, 2020.

[34] C. Deng, X. Liu, C. Li, and D. Tao, "Active multi-kernel domain adaptation for hyperspectral image classification," *Pattern Recognit.*, vol. 77, pp. 306–315, May 2018.

[35] H. L. Yang and M. M. Crawford, "Domain adaptation with preservation of manifold geometry for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 2, pp. 543–555, Feb. 2016.

[36] Z. Wang, B. Du, Q. Shi, and W. Tu, "Domain adaptation with discriminative distribution and manifold embedding for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 7, pp. 1155–1159, Jul. 2019.

[37] L. Jiao, M. Liang, H. Chen, S. Yang, H. Liu, and X. Cao, "Deep fully convolutional network-based spatial distribution prediction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5585–5599, Oct. 2017.

[38] J. Yang, Y.-Q. Zhao, and J. C.-W. Chan, "Learning and transferring deep joint spectral–spatial features for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4729–4742, Aug. 2017.

[39] S. Mei, J. Ji, J. Hou, X. Li, and Q. Du, "Learning sensor-specific spatial–spectral features of hyperspectral images via convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4520–4533, Aug. 2017.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.

[40] N. Yokoya and A. Iwasaki, "Airborne hyperspectral data over Chikusei," Space Appl. Lab., Univ. Tokyo, Tokyo, Japan, Tech. Rep. SAL-2016-05-27, May 2016.
[41] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
[42] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.
[43] L. Wang, S. Hao, Q. Wang, and Y. Wang, "Semi-supervised classification for hyperspectral imagery based on spatial–spectral label propagation," *ISPRS J. Photogram. Remote Sens.*, vol. 97, pp. 123–137, Nov. 2014.



Jing Bai (Senior Member, IEEE) received the B.S. degree in electronic and information engineering from Zhengzhou University, Zhengzhou, China, in 2004, and the Ph.D. degree in pattern recognition and intelligent systems from Xidian University, Xi'an, China, in 2009.

She is currently an Associate Professor with Xidian University. Her research interests include image processing, machine learning, and intelligent information processing.



Shaojie Huang received the B.S. degree in electronic information engineering from Shantou University, Shantou, China, in 2019. He is currently pursuing the M.S. degree in computer science and technology with Xidian University, Xi'an, China.

He is interested in image processing, deep learning, and intelligent information processing.



Zhu Xiao (Senior Member, IEEE) received the M.S. and Ph.D. degrees in communication and information systems from Xidian University, Xi'an, China, in 2007 and 2009, respectively.

From 2010 to 2012, he was a Research Fellow with the Department of Computer Science and Technology, University of Bedfordshire, Luton, U.K. He is currently an Associate Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include wireless localization, the Internet of Vehicles, and intelligent transportation systems.

Dr. Xiao is serving as an Associate Editor for IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS.



Xianmin Li received the B.S. degree in intelligent science and technology and the M.S. degree in computer science and technology from Xidian University, Xi'an, China, in 2018 and 2021, respectively.

He is interested in image processing, computer vision, deep learning, and intelligent information processing.



Yongdong Zhu received the bachelor's degree from Xi'an Jiaotong University, Xi'an, China, in 1997, and the Ph.D. degree from the University of Essex, Colchester, U.K., in 2007.

He is a Professor with the Institution of Intelligent Network, Zhejiang Lab, Hangzhou, Zhejiang, China. His research interests include the next-generation mobile communication, mobile edge network architecture, vehicular communication networks, and the Internet of Things.



Amelia C. Regan (Member, IEEE) received the M.S. degree in mathematics from Johns Hopkins University, Baltimore, MD, USA, and the M.S.E. and Ph.D. degrees from The University of Texas at Austin, Austin, TX, USA.

She is a Professor with the Donald Bren School of Information and Computer Sciences, University of California at Irvine, Irvine, CA, USA. Her research interests include cyber-physical transportation systems, dynamic and stochastic network optimization, and machine learning tools for temporal–spatial data analysis.



Licheng Jiao (Fellow, IEEE) received the B.S. degree from Shanghai Jiao Tong University, Shanghai, China, in 1982, and the M.S. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 1984 and 1990, respectively.

Since 1992, he has been a Professor with the School of Electronic Engineering, Xidian University, Xi'an, where he is currently the Director of the Key Laboratory of Intelligent Perception and Image Understanding of the Ministry of Education, International Research Center of Intelligent Perception and Computation. His research interests include intelligent information processing, image processing, machine learning, and pattern recognition.

Dr. Jiao is a member of the IEEE Xi'an Section Execution Committee, the President of the Computational Intelligence Chapter, the IEEE Xi'an Section, and the IET Xi'an Network, the Chairperson of the Awards and Recognition Committee, the Vice Board Chairperson of the Chinese Association of Artificial Intelligence, a Councilor of the Chinese Institute of Electronics, a Committee Member of the Chinese Committee of Neural Networks, and an Expert of the Academic Degrees Committee of the State Council.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 20:54:51 UTC from IEEE Xplore. Restrictions apply.